# Data Driven Control Integration Week

## Group 8 (MPC)

Christian Brunet  Jules Marceau
Matthieu Moussaoui  Clément Tamboise
Rayane Trigoust

CentraleSupélec

Friday, November 6, 2026

# Table of Contents


1. Introduction
2. Dataset Validation
3. System Modeling
4. First RNN
5. Z-axis control
6. Advanced Control Law

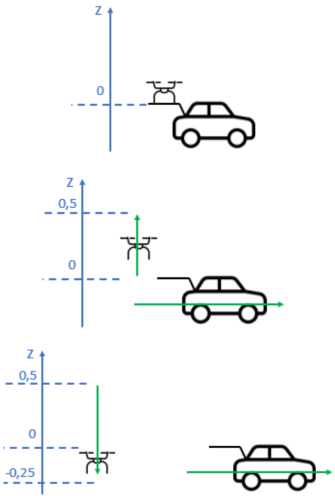
# Project Objectives

**Available Resources:**

- Dataset of vertical acceleration.
- MATLAB/Simulink drone simulator.

**Objectives:**

- Validate the dataset.
- Identify the model.
- Develop a control law.



# Dataset Validation

## Cleaning the data

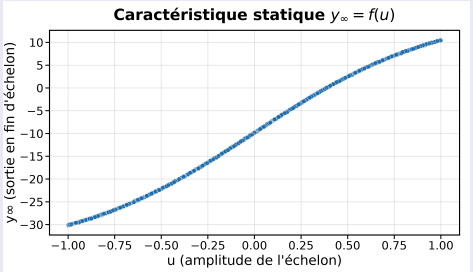
**Remove lines that includes:**

- Frozen sensor for 2+ seconds
- 5+ diverging values

## Normalisation

- Directly in the training

## Static characteristic




$y_\infty = f(u)$ after eliminating corrupted runs.

# System Modeling

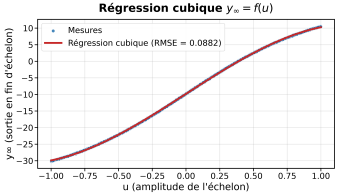
## Linear Model: Hammerstein Approach

### Static Characteristic $N(u)$

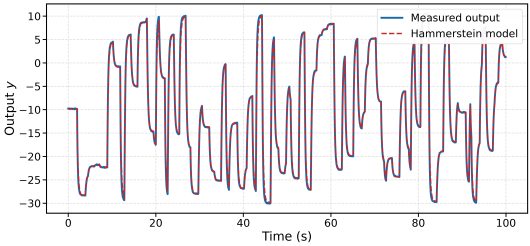
3rd-order polynomial identified on steady-state regimes:

$$v(t) = N(u) = a_3 u^3 + a_2 u^2 + a_1 u + a_0$$

**Tau identification**




**Hammerstein Model Validation**




**Least-Squares Linear Model Validation (a=0.7836, b=4.9676, c=-2.1222)**

# System Modeling

## Linear Model: Least Square Method

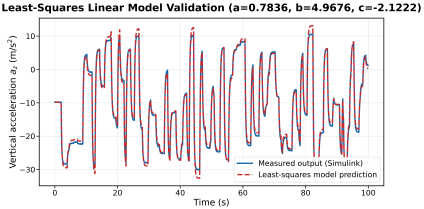
### Model equations

$$y_k(t+1) = a\,y_k(t) + b\,u_k(t) + c$$
$$= \varphi_k^{\mathsf{T}}\theta.$$

$$\varphi_k^{\mathsf{T}} = \begin{bmatrix} y_k(t) & u_k(t) & 1 \end{bmatrix} \qquad \theta = \begin{bmatrix} a & b & c \end{bmatrix}^{\mathsf{T}}.$$

The least-squares estimate is

$$\theta^{\star} = \left(\Phi^{\mathsf{T}}\Phi\right)^{-1}\Phi^{\mathsf{T}}Y.$$



# First RNN

## Sequence-to-sequence architecture

### Parameters

- One scalar input $u_t$ and one scalar output $\hat{y}_t$
- Hidden state dimension: 16
- Early stopping to avoid overfitting

### State representation

$$h_t = \tanh(W_{ih}u_t + W_{hh}h_{t-1} + b)$$

$$\hat{y}_t = W_{hy}h_t + b_y$$

# First RNN

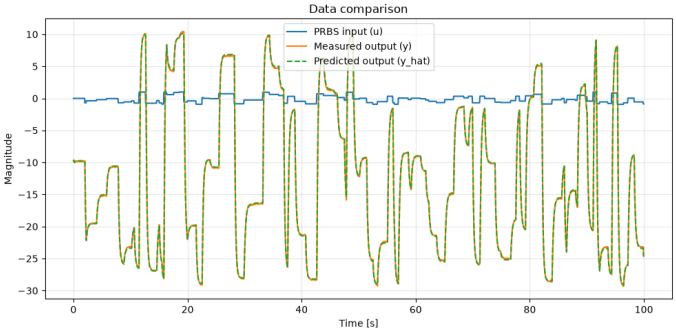
## Results

### Numerical results

- Validation loss: $3.93 \times 10^{-4}$
- Test loss: $4.01 \times 10^{-4}$

### Training criterion

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$



# Z-axis control

## Designing the command

### Principle

- Python Class Wrapping
- Simulink Signal Generation
- LQ control for the linear model
- MPC control for the RNN

### Results

Add here the main result or performance metric.

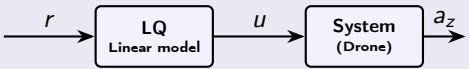
### Figure

*Insert figure here*

# LQ control

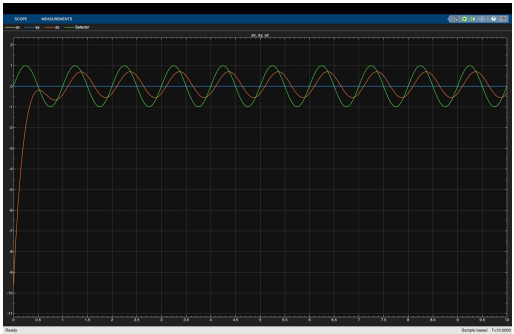
## Linear-quadratic control design

### Control principle




### Performance

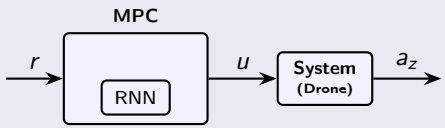
- Global trajectory following
- Wrong amplitude + phase shift
- Missing prediction



# MPC control

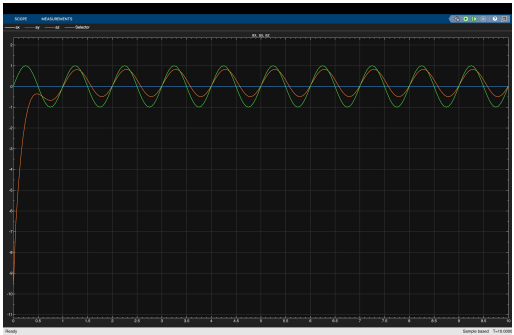
## Model predictive control design

### Control principle




### Performance

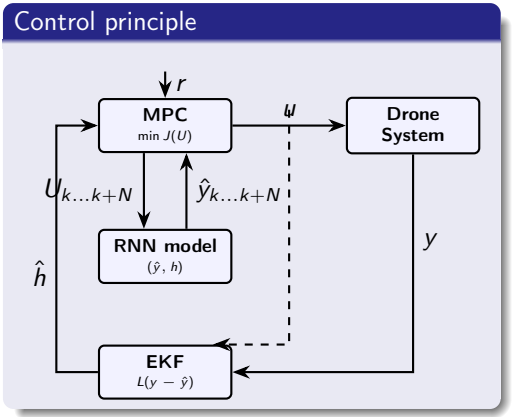
- Good overall tracking.
- No phase shifting good prediction.
- Full amplitude not reached.
- Only internal state.

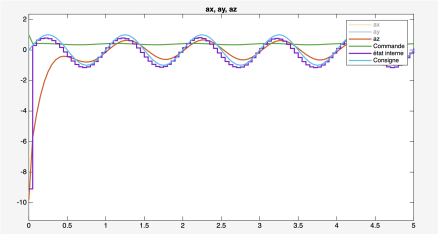


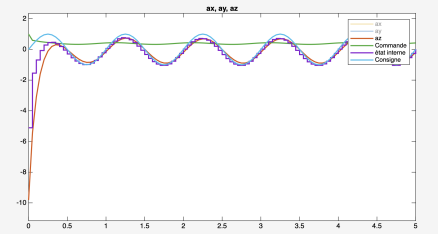
# MPC with EKF

State estimation and predictive control

## Control principle



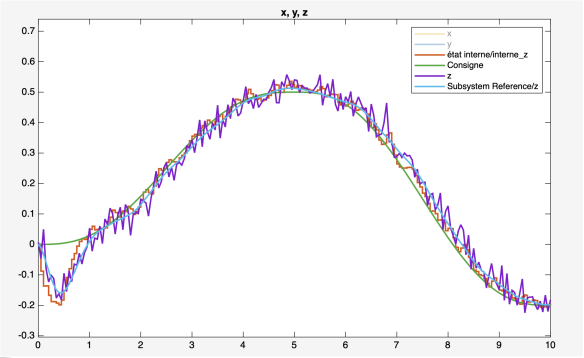
# Requested command

## MPC control with EKF feedback

### Control objective

The EKF filters the noisy position measurement before it is used by the predictive controller.

- State estimation from noisy measurements.
- Improved feedback for position tracking.
- More robust closed-loop response.




### Performance

Position tracking remains stable despite measurement noise.

# Comparison

## LQ, MPC and MPC with EKF

### LQ control with Linear Model

- Simple and very fast.
- Lack prediction.
- Phase shifting.

### MPC control with RNN

- Prediction.
- Internal state diverges from reality.
- No feedback loop.

### MPC with EKF

- Feedback from measurements.

# Data generation

PRBS excitation in the 3D drone workspace

## Objective

Generate sufficiently rich input/output trajectories to identify a nonlinear dynamic model of the drone.

- Excite the four actuator commands around hover
- Capture the coupled 3D dynamics (12 variables)
- Store synchronized inputs and measured outputs

## Simulation setup

- Sampling time: $T_s = 0.05\,\mathrm{s}$

## APRBS modes

We don't directly generate u inputs (unusable data).
Four independently generated piecewise-constant modes:

$$\delta = [\delta_T, \delta_r, \delta_p, \delta_y]$$

Each level is sampled uniformly and held for a random duration between 1 and 3 seconds.

## Initial phase

The excitation starts after $t_{\mathrm{start}} = 2\,\mathrm{s}$, allowing the drone to stabilize at hover.

# Data generation

## From modal excitation to motor commands

### Mixer matrix

The four modal perturbations are mixed into the actuator commands:

$$U = U_{\text{hover}} + \delta \begin{bmatrix} 1 & 1 & -1 & 1 \\ 1 & -1 & -1 & -1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 1 & -1 \end{bmatrix}^T$$

The resulting commands are clipped to the allowed range.

### Input/output data

- Inputs: $U_1, \ldots, U_4$
- Outputs: angles, angular velocities, accelerations and angular accelerations
- One trajectory is stored as a sequence of $N$ time steps
- Different random seeds improve excitation diversity